

Terraform & Atlantis Operational Standards

Nexlify Technologies | Platform Engineering | NX-OPS-TF-003 | Rev 5.1

1. Purpose & Tooling Stack

All AWS infrastructure for Nexlify is declared in Terraform 1.5+ and stored in the GitHub organisation nexlify-infra. Atlantis (self-hosted on the shared-services EKS cluster) is the exclusive plan/apply engine. Direct terraform apply from developer laptops is forbidden for any workspace that manages production or shared resources. Atlantis listens for pull-request comments containing 'atlantis plan' and 'atlantis apply'. State files live in S3 (s3://nexlify-tfstate-ap-south-1) with DynamoDB locking (table nexlify-tf-locks). Backend configuration is injected by Atlantis; developers never hard-code credentials.

2. Repository Layout & Module Versioning

Root modules are organised by environment and account: environments/prod/, environments/nonprod/, environments/shared/. Reusable modules live under modules/ and are versioned with semantic tags (vX.Y.Z). Consuming root modules pin module sources to a specific tag; floating 'main' is rejected by the Atlantis pre-workflow hook. The private Terraform module registry (hosted on Artifactory) mirrors approved external modules after security scanning. Every module must expose an outputs.tf that documents the values needed by downstream consumers. Breaking changes require a major version bump and a migration guide in the PR.

3. Atlantis Workflow Configuration

The atlantis.yaml in each repository defines projects with explicit dir, workspace, and autoplan settings. Autoplan is enabled for pull requests that touch .tf files; apply requires at least one approval from a CODEOWNER outside the authoring team. Parallel plans are allowed across independent projects; sequential apply is enforced when projects share state. A custom workflow runs 'terraform fmt -check', 'tflint', and 'checkov' before the official plan. Failed policy checks block the plan comment. Apply is restricted to the main branch after merge; feature-branch applies are disabled for production workspaces.

4. State Locking, Drift Detection & Secrets

DynamoDB state locking prevents concurrent modifies. Atlantis holds the lock for the duration of plan and apply; orphaned locks older than 2 hours are cleaned by a scheduled Lambda. Drift detection runs nightly via Atlantis 'atlantis plan -d' against the main branch for every production workspace; results are posted to #platform-drift Slack channel. Secrets are never stored in Terraform state: sensitive values are read from AWS Secrets Manager or SSM Parameter Store using the aws_secretsmanager_secret_version data source with the sensitive flag. The Atlantis service account assumes an IAM role that has least-privilege access scoped by path prefix.

5. Change Management & Rollback

Every infrastructure change that affects production must reference a Jira ticket of type Change. The PR description template requires risk assessment, rollback plan, and affected services. After apply, Atlantis posts the plan summary to the ticket. Rollback is performed by reverting the Git commit and re-running atlantis apply; for irreversible changes (e.g., RDS major version upgrade) a documented manual procedure is required. The Platform team maintains a canary workspace that mirrors production topology at reduced scale; high-risk changes are first applied there.

6. Operational Commands Reference

In a pull request comment: 'atlantis plan -p ' to plan a specific project; 'atlantis apply -p ' after approval; 'atlantis unlock' to release a stuck lock. For local debugging (read-only): terraform init with the remote backend, then terraform plan -lock=false. Never use terraform destroy on production without explicit CAB approval. Module upgrades are performed by updating the source version and running plan; Atlantis will surface any resource replacement.

Owner: Platform Engineering | Slack: #atlantis-ops | Runbook location: Confluence/PlatformOps/Terraform | Last updated 2026-07-01.